

fxml之一



FXML基础

北京理工大学计算机学院
金旭亮

JavaFX视图与FXML



在采用MVC架构的JavaFX应用中，“视图（View）”是一种XML格式的文本文件，以声明的方式定义了若干个UI控件，组成一个控件树，构建出整个用户界面。



JavaFX视图的设计语言，称为“FXML”，是一种基于XML的标记语言，主要用于构建JavaFX应用的UI界面，实现UI代码与功能代码的分离。

FXML文档的结构

xml 声明

</> main-view.fxml x

```
1  <?xml version="1.0" encoding="UTF-8"?>
2
3  <?import javafx.geometry.Insets?>
4  <?import javafx.scene.control.Label?>
5  <?import javafx.scene.layout.VBox?>
6  <?import javafx.scene.control.Button?>
7
8  <VBox alignment="CENTER" spacing="20.0"
9      xmlns:fx="http://javafx.com/fxml"
10     fx:controller="com.jinxuliang.fxmlbasic.MainController">
11    <padding>
12      <Insets bottom="20.0" left="20.0" right="20.0" top="20.0"/>
13    </padding>
14    <Label fx:id="welcomeText"/>
15    <Button text="Hello!" />
16  </VBox>
```

类型导入声明

FXML声明的控件树

JavaFX项目中的fxml文档

Project ▾

▾  **FXMLBasic** D:\JavaProjects\FXMLBasic

>  .idea

>  .mvn

▾  src

▾  main

▾  java

▾  com.jinxuliang.fxmlbasic

>   FXMLBasicApplication

  MainController

 module-info.java

▾  resources

▾  com.jinxuliang.fxmlbasic

  main-view.fxml

>  target

 .gitignore

 mvnw

 mvnw.cmd

 pom.xml

>  External Libraries

>  Scratches and Consoles

@Override

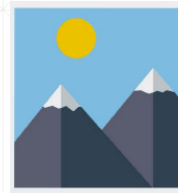
```
public void start(Stage stage) throws IOException {  
    FXMLLoader fxmlLoader = new FXMLLoader(  
        FXMLBasicApplication.class.getResource("main-view.fxml"));  
    Scene scene = new Scene(fxmlLoader.load());  
    stage.setTitle("FXML基础");  
    stage.setScene(scene);  
    stage.show();  
}
```

通常会将fxml放到资源文件夹中以资源的方式加载。

使用FXMLLoader()装载FXML文档，文档中的根元素，成为实例化后控件树的根节点。

注意fxml在资源文件夹中的位置，推荐与用到它的控制器所在的包名保持一致，这样可以简化开发。

FXML元素与JavaFX控件



JavaFX视图文件（FXML文档）主要由元素（element）组成，通常对应着特定的JavaFX控件。当JavaFX程序运行时，JavaFX框架读入FXML文件，解析它，将相应的FXML元素转换为特定JavaFX控件对象，并依据XML元素的嵌套关系，将这些控件对象装配为一棵控件树，把根节点传给Scene，成为“场景图（Scene Graph）”。



在FXML文档中使用JavaFX控件，要先导入，再使用：

```
<?import javafx.scene.control.Label?>  
  
<Label text="Hello, World!"/>
```

FXML元素与JavaFX控件的对应关系

大写字母开头的
元素名



JavaFX对象实例

```
<?import javafx.scene.control.VBox?>  
  
<VBox ...>  
    ...  
</VBox>
```

事实上，除了JavaFX控件，在FXML文档中还可以放置其它类型的Java对象，后面会看到相应的例子。



类型要先导入，再使用。

命名空间与id值

```
<VBox xmlns:fx="http://javafx.com/fxml">  
    <Label fx:id="label1" text="Line 1"/>  
</VBox>
```

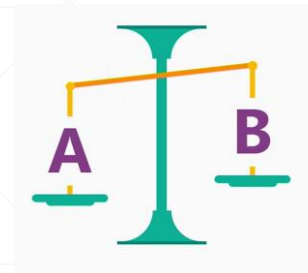


在同一个XML命名空间内，id值应该唯一。



在JavaFX控制器中，id值被用来引用控件。

FXML文档关联控制器的两种方式



1 声明式绑定方式

```
<VBox xmlns:fx="http://javafx.com/fxml"
      fx:controller="com.jinxuliang.MyFXMLController" >
    ...
</VBox>
```

采用这种方式，控制器实例可以通过以下代码获取：

```
MyFXMLController controllerRef = loader.getController();
```

2 使用代码设定：

```
MyFXMLController controller = new MyFXMLController();

FXMLLoader loader = new FXMLLoader();
loader.setController(controller);
```


FXML文档的组合

可以使用<fx:include> “组合” 多个FXML文档为一个文档。

```
<fx:include source="FXML文档名"/>
```

示例：

main-view.fxml

```
<VBox alignment="CENTER" spacing="20.0"
  xmlns:fx="http://javafx.com/fxml"
  fx:controller="com.jinxuliang.fxmlbasic.MainController">
  <padding>
    <Insets bottom="20.0" left="20.0" right="20.0" top="20.0"/>
  </padding>
  <Label fx:id="lblInfo"/>
  <fx:include fx:id="mybutton" source="my-button.fxml"/>
</VBox>
```

my-button.fxml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <?import javafx.scene.control.*?>
3 <Button text="My Button"/>
```

使用导入的按钮控件

```
public class MainController implements Initializable {  
    2 usages  
    @FXML  
    private Label lblInfo;  
    2 usages  
    @FXML  
    private Button mybutton;  
    @Override  
    public void initialize(URL url, ResourceBundle resourceBundle) {  
        mybutton.setOnAction(e->{  
            lblInfo.setText("当前时间: "+ LocalDateTime.now());  
        });  
    }  
}
```

字段名与id值一致



完整的FXML技术文档

https://openjfx.io/javadoc/19/javafx.fxml/javafx/fxml/doc-files/introduction_to_fxml.html

JavaFX网站主页



Reference Documentation

- Getting Started with JavaFX 11+
- API documentation
- [Introduction to FXML](#)
- JavaFX CSS Reference Guide
- Release Notes

ORACLE



Introduction to FXML

Last updated: 01 May 2017

Contents

- Overview
- Elements
 - Class Instance Elements
 - Instance Declarations
 - `<fx:include>`
 - `<fx:constant>`
 - `<fx:reference>`
 - `<fx:copy>`
 - `<fx:root>`